

Звіт

до Лабораторної роботи №8

Файлове введення-виведення, серіалізація та винятки

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Мета роботи

Метою лабораторної роботи є отримання практичних навичок роботи з файловим введенням і виведенням у Java, текстовим збереженням даних, бінарною серіалізацією об'єктів та обробкою винятків.

У роботі потрібно реалізувати предметну модель, клас-контейнер, збереження даних у текстовий файл, відновлення даних із текстового файлу, серіалізацію у бінарний файл, десеріалізацію та користувацький виняток для помилкових ситуацій.

2. Опис предметної області

У лабораторній роботі розглядається предметна область засобів командної роботи над проектами.

У такій системі можуть існувати різні елементи робочого простору: картка завдання, звіт про помилку та примітка до рев'ю. Для кожного елемента потрібно зберігати основну інформацію: ідентифікатор, назву, виконавця, автора, текст, рівень важливості або кількість коментарів.

Для збереження набору таких об'єктів використовується клас-контейнер ProjectWorkspace.

3. Структура проєкту

У роботі використано розділення коду за пакетами. Це дозволяє не змішувати класи моделі, логіку контейнера, файловий доступ, винятки та демонстраційний запуск.

Пакет	Призначення
model	Класи предметної області.
service	Клас-контейнер для зберігання об'єктів.
persistence	Клас для текстового збереження, читання, серіалізації та десеріалізації.
exception	Користувацький виняток предметної області.
demo	Демонстраційна програма Main.

4. Класи предметної області

4.1 Клас TaskCard

Клас TaskCard описує картку завдання у робочому просторі проєкту.

Поля класу:

- id — ідентифікатор завдання;
- title — назва завдання;
- assignee — виконавець;
- commentsCount — кількість коментарів.

Клас реалізує Serializable, тому його об'єкти можна зберігати у бінарному файлі через механізм серіалізації. Також клас має метод toTextLine(), який перетворює об'єкт у текстовий рядок для збереження у файлі.

4.2 Клас BugReport

Клас BugReport описує звіт про помилку.

Поля класу:

- id — ідентифікатор помилки;
- severity — рівень важливості помилки;
- assignee — відповідальний виконавець;
- commentsCount — кількість коментарів.

Клас також реалізує Serializable і має метод toTextLine() для текстового збереження.

4.3 Клас ReviewNote

Клас ReviewNote описує примітку до рев'ю коду.

Поля класу:

- id — ідентифікатор примітки;
- author — автор примітки;
- text — текст примітки;
- commentsCount — кількість коментарів.

Клас реалізує Serializable, тому його об'єкти можуть бути серіалізовані разом з усім робочим простором.

5. Клас-контейнер ProjectWorkspace

Клас ProjectWorkspace є контейнером для зберігання елементів робочого простору проекту.

У ньому зберігаються три колекції:

- List<TaskCard> — список завдань;
- List<BugReport> — список помилок;
- List<ReviewNote> — список приміток рев'ю.

Основні методи класу:

- addTask(...) — додавання завдання;
- addBug(...) — додавання помилки;
- addReviewNote(...) — додавання примітки;

- `findTaskById(...)` — пошук завдання за ідентифікатором;
- `findBugById(...)` — пошук помилки за ідентифікатором;
- `findReviewNoteById(...)` — пошук примітки за ідентифікатором;
- `getTotalCount()` — отримання загальної кількості елементів;
- `showAll()` — виведення всіх елементів у текстовому вигляді.

Клас `ProjectWorkspace` реалізує `Serializable`, оскільки серіалізується весь контейнер разом із колекціями об'єктів.

6. Користувацький виняток

У роботі створено власний виняток:

`InvalidWorkspaceDataException`

Цей виняток використовується для помилок, пов'язаних із некоректними даними робочого простору.

Він може виникати у таких ситуаціях:

- дублювання ідентифікатора завдання;
- дублювання ідентифікатора помилки;
- дублювання ідентифікатора примітки;
- неправильна кількість полів у текстовому файлі;
- невідомий тип запису у текстовому файлі;
- некоректний числовий формат.

Це дозволяє відокремити логічні помилки предметної області від звичайних помилок введення-виведення.

7. Текстовий формат збереження

Для текстового збереження використовується простий формат: один об'єкт — один рядок. Поля розділяються символом крапки з комою ;.

Перше поле визначає тип запису:

- `TASK` — картка завдання;
- `BUG` — звіт про помилку;
- `REVIEW` — примітка до рев'ю.

Клас	Формат	Приклад
TaskCard	TASK;id;title;assignee;commentsCount	TASK;TASK-1;Створити сторінку входу;Колісниченко Артем;2
BugReport	BUG;id;severity;assignee;commentsCount	BUG;BUG-101;Висока;Іван Петренко;1
ReviewNote	REVIEW;id;author;text;commentsCount	REVIEW;NOTE-1;Артем;Код потребує невеликих виправлень;3

Усі рядки повинні мати рівно 5 полів. Якщо кількість полів неправильна, програма генерує `InvalidWorkspaceDataException`.

8. Клас `WorkspaceFileManager`

Клас `WorkspaceFileManager` відповідає за файлове введення-виведення.

Він містить такі методи:

- `saveAsText(...)` — збереження робочого простору у текстовий файл;
- `loadFromText(...)` — читання робочого простору з текстового файлу;
- `serialize(...)` — серіалізація контейнера у бінарний файл;
- `deserialize(...)` — десеріалізація контейнера з бінарного файлу.

Метод `saveAsText(...)` зберігає всі об'єкти з `ProjectWorkspace` у текстовий файл.

Метод `loadFromText(...)` читає текстовий файл, визначає тип кожного рядка та відновлює відповідні об'єкти.

Метод `serialize(...)` записує весь об'єкт `ProjectWorkspace` у бінарний файл.

Метод `deserialize(...)` читає бінарний файл і відновлює об'єкт `ProjectWorkspace`.

Під час роботи з файлами використовується конструкція `try-with-resources`, яка забезпечує автоматичне закриття файлових потоків.

9. Серіалізація і десеріалізація

У роботі реалізовано бінарне збереження стану програми через стандартний механізм `Java Serialization`.

Для цього класи `TaskCard`, `BugReport`, `ReviewNote` і `ProjectWorkspace` реалізують інтерфейс `Serializable`.

Серіалізація виконується за допомогою `ObjectOutputStream`. Десеріалізація виконується за допомогою `ObjectInputStream`.

На відміну від текстового формату, серіалізація зберігає весь контейнер як об'єктну структуру. Це зручно для швидкого відновлення стану програми, але такий файл не призначений для читання людиною.

10. Обробка винятків

У програмі обробляються такі типи помилок:

- `IOException` — помилки введення-виведення;
- `ClassNotFoundException` — помилки під час десеріалізації;
- `InvalidWorkspaceDataException` — помилки даних робочого простору.

У класі `Main` передбачено блоки `catch`, які виводять зрозумілі повідомлення користувачу.

Приклад помилкової ситуації:

```
TASK;ONLY_TWO_FIELDS
```

Цей рядок має неправильну кількість полів. Під час читання такого файлу програма генерує `InvalidWorkspaceDataException` з повідомленням про некоректну кількість полів.

11. UML-діаграма

Для моделі було створено UML-діаграму. На ній показано класи предметної області, клас-контейнер, клас файлового збереження та користувацький виняток.

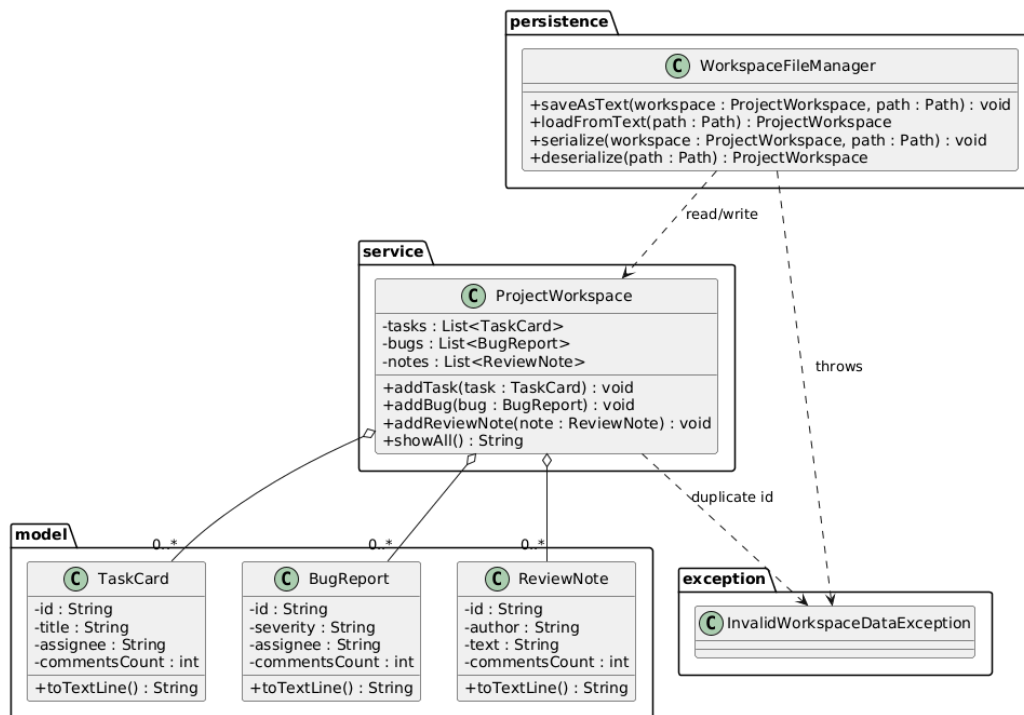


Рисунок 1 — UML-діаграма моделі робочого простору проєкту

12. Демонстрація роботи програми

У класі Main демонструється така послідовність дій:

1. Створення об'єкта ProjectWorkspace.
2. Додавання TaskCard, BugReport і ReviewNote.
3. Виведення початкового робочого простору.
4. Пошук завдання за ідентифікатором.
5. Збереження даних у текстовий файл.
6. Завантаження даних із текстового файлу.
7. Серіалізація у бінарний файл.
8. Десеріалізація з бінарного файлу.
9. Демонстрація помилкової ситуації з неправильним форматом рядка.

Після запуску програми створюються файли:

build/lab08/workspace.txt

build/lab08/workspace.bin

build/lab08/broken-workspace.txt

```
Завдання: TASK-1, назва: Створити сторінку входу, виконавець: Колісниченко Артем, коментарів: 2
=== Помилки ===
Помилка: BUG-101, важливість: Висока, виконавець: Іван Петренко, коментарів: 1
=== Примітки рев'ю ===
Примітка рев'ю: NOTE-1, автор: Артем, текст: Код потребує невеликих виправлень, коментарів: 3

Пошук завдання TASK-1:
Завдання: TASK-1, назва: Створити сторінку входу, виконавець: Колісниченко Артем, коментарів: 2

Дані збережено у текстовий файл: build\lab08\workspace.txt

Дані, відновлені з текстового файлу
=== Завдання ===
Завдання: TASK-1, назва: Створити сторінку входу, виконавець: Колісниченко Артем, коментарів: 2
=== Помилки ===
Помилка: BUG-101, важливість: Висока, виконавець: Іван Петренко, коментарів: 1
=== Примітки рев'ю ===
Примітка рев'ю: NOTE-1, автор: Артем, текст: Код потребує невеликих виправлень, коментарів: 3

Дані серіалізовано у бінарний файл: build\lab08\workspace.bin

Дані, відновлені з бінарного файлу
=== Завдання ===
Завдання: TASK-1, назва: Створити сторінку входу, виконавець: Колісниченко Артем, коментарів: 2
=== Помилки ===
Помилка: BUG-101, важливість: Висока, виконавець: Іван Петренко, коментарів: 1
=== Примітки рев'ю ===
Примітка рев'ю: NOTE-1, автор: Артем, текст: Код потребує невеликих виправлень, коментарів: 3

Демонстрація помилкової ситуації
Помилка даних робочого простору: Invalid field count at line 1
```

Рисунок 2 — результат запуску програми

13. Тестування

Для перевірки роботи програми були створені тести JUnit Jupiter.

Тестові класи:

- TaskCardTest;
- BugReportTest;
- ReviewNoteTest;
- ProjectWorkspaceTest;
- WorkspaceFileManagerTest.

Тести перевіряють:

- створення об'єктів предметної області;

- правильність збереження полів;
- перетворення об'єктів у текстовий рядок;
- заборону некоректних значень;
- додавання об'єктів у ProjectWorkspace;
- пошук об'єктів за ідентифікатором;
- заборону дублювання ідентифікаторів;
- збереження і читання текстового файлу;
- серіалізацію і десеріалізацію;
- реакцію на пошкоджений текстовий файл.

```
[INFO] Running ua.khpi.oop.lab08.model.BugReportTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in ua.khpi.oop.lab08.model.BugReportTest
[INFO] Running ua.khpi.oop.lab08.model.ReviewNoteTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in ua.khpi.oop.lab08.model.ReviewNoteTest
[INFO] Running ua.khpi.oop.lab08.model.TaskCardTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.006 s -- in ua.khpi.oop.lab08.model.TaskCardTest
[INFO] Running ua.khpi.oop.lab08.persistence.WorkspaceFileManagerTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.081 s -- in ua.khpi.oop.lab08.persistence.WorkspaceFileManagerTest
[INFO] Running ua.khpi.oop.lab08.service.ProjectWorkspaceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.005 s -- in ua.khpi.oop.lab08.service.ProjectWorkspaceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
```

Рисунок 3 — результат виконання тестів

14. Додаткове завдання

Для додаткового завдання можна розглянути попередню модель спортивного клубу.

У цій моделі були класи:

- Student;
- Coach;
- SportSection;
- Schedule;
- Enrollment;
- SportsClub.

Таку модель також можна адаптувати до зовнішнього збереження даних.

Наприклад, для секцій можна використати текстовий формат:

SECTION;id;name;coachName;day;time;available

Для студентів:

STUDENT;id;fullName;group

Для запису студента на секцію:

ENROLLMENT;studentId;sectionId;date;active

Обов'язковими полями були б ідентифікатори, назви, ПІБ студента або тренера, дата запису та статус активності.

Можливі помилки формату:

- неправильна кількість полів;
- порожній ідентифікатор;
- дублювання запису;
- некоректна дата;
- невідомий тип запису;
- посилання на неіснуючого студента або секцію.

Серіалізація всієї колекції для такої моделі також є доцільною, тому що SportsClub може виступати контейнером для всіх студентів, секцій і записів.

15. Висновок

У ході виконання лабораторної роботи було реалізовано програму для обліку елементів командної роботи над проектом.

Було створено три класи предметної області: TaskCard, BugReport і ReviewNote.

Також було реалізовано клас-контейнер ProjectWorkspace, який зберігає списки завдань, помилок і приміток рев'ю.

У роботі реалізовано текстове збереження даних у файл, читання даних із текстового файлу, серіалізацію у бінарний файл та десеріалізацію.

Для обробки помилок створено власний виняток InvalidWorkspaceDataException.

Під час роботи з файлами використано try-with-resources, що забезпечує правильне закриття файлових потоків.

Таким чином, лабораторна робота показала, як можна зберігати стан об'єктної моделі за межами оперативної пам'яті та відновлювати його під час наступного запуску програми.